

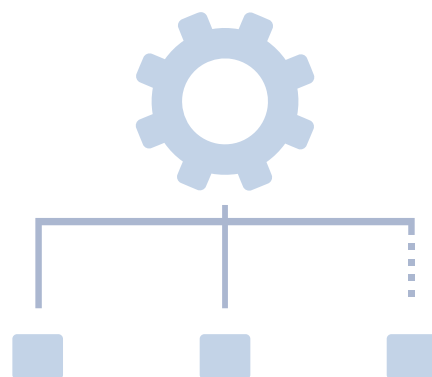


North South University

HW : 09

SUBMITTED BY :

Mohammad Tushar Abdullah
2323766042



HW-09

MOHAMMAD TUSHAR
ABDULLAH
IP- 2323766042

Ans. to the question no-01

④ List based on arrays:

① contains:

Time complexity: $O(n)$

② Insert:

Time complexity: $O(1)$

④ Linked List:

① contains:

Time complexity: $O(n)$

② Insert:

Time complexity:

with tail: $O(1)$

without tail: $O(n)$

④ Trees:

① contain:

Time complexity: $O(\log n)$

② Insert:

Time complexity: $O(\log n)$

⑤ Hash table:

① contain:

Time complexity: $O(n)$

② Insert:

Time complexity: $O(1)$

Ans. to the question-02

(a) Given numbers: 12, 15, 3, 35, ~~31~~ 21, 42, 14

insert 12:

12

insert 15:

12
 \
 15

insert 3:

12
/
3 \
 15

insert 35:

12
/
3 \
 15 \
 35

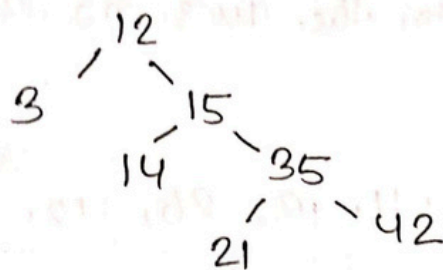
insert 21:

12
/
3 \
 15 \
 35
 /
 21

insert 42:

12
/
3 \
 15 \
 35
 /
 21 \
 42

insert 14:



(b) Given numbers: 12, 15, 3, 35, 21, 42, 14

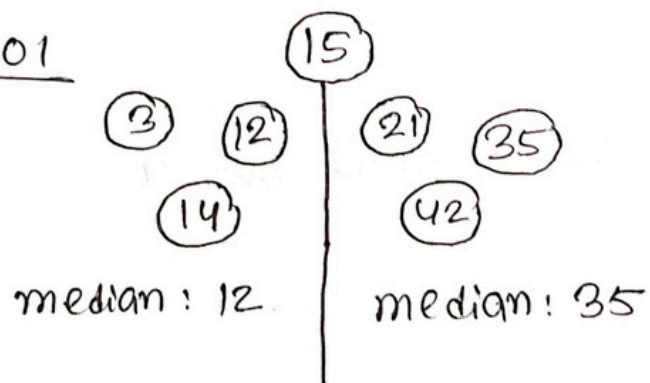
in sorted order: 3, 12, 14, 15, 21, 35, 42

median: 15 so root: 15.

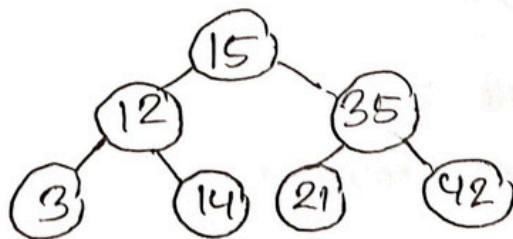
now, 3, 12, 14 at the left of 15 and 21,

35, 42 are at the right of 15

step 01



step 02



This is the balanced binary search tree.

Ans. to the ques. no-03

Pre order: 30, 24, 11, 13, 26, 40, 58, 48.

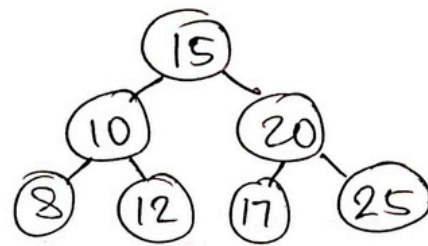
Post order: 13, 11, 26, 24, 48, 58, 40, 30

In order: 11, 13, 24, 26, 30, 40, 48, 58

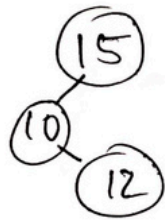
For a BST, the inorder traversal always prints the keys in sorted order because the left subtree contains smaller element and right one contains larger ~~element~~ elements.

(4)

Given tree,



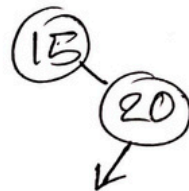
① Find (12, R)



Nodes visited: 15, 10, 12

Find output: 12

② Find (18, R)



Nodes visited: 15, 20

key not in tree

③ Time complexity in worst case $O(\log n)$.

Modified algorithm :

Function find (k, R):

if R is ~~not~~ null :

return "key not found"

if $R.key == k$:

return R

else if $R.key > k$:

return find ($k, R.Left$)

else

return find ($k, R.Right$)

Ans. to the question - ~~15~~ 05

